

CS204 – Algorithms & Data Structures Laboratory

Dr. V. Vijaya Saradhi,
Dept. Of CSE,
IIT Guwahati

saradhi@iitg.ac.in



Tutorial #04 C++ Operator Overloading



Overview



- **Introduction**
- **Examples**
- **Overloading binary operators**
- **Overloading unary operators**
- **Overloading involving only objects**
- **Overloading involving object and built-in data type**
- **Overloading involving two objects from different classes**

Introduction



- So far you have used function names such as `compute_gx`, `compute_gy`, `forward_elimination` etc
- The function naming convention is identical to variable naming convention
- In operator overloading, function names can be symbols from built-in list of operators
- That is one can name function as "+"
- Function name can be "*"
- Function name can be "++" etc

Introduction



- Where would one use functions whose name is an operator?
- They are used specifically for user defined data types
- C++ does not allow new operators to be created
- However, existing set of operators can be re-used with objects as long as they have meaning
- Example, `m1` and `m2` are objects of type `Matrix`
- The statement `Matrix m3 = m1 + m2` would not make any sense if it were a standard `+` operation
- If `+` operation is defined on matrices the name would make absolute sense.

Introduction



- The statement `Matrix m3 = m1 + m2` will not work automatically
- You must write operator overloading function "+" defined on two matrix objects
- An operator is overloaded by writing a member function definition
- An operator is overloaded by writing a non-member function definition
- Function name starts with the keyword **operator** followed by the symbol operator being overloaded
- Example: **operator +**
- Example: **operator -**
- Example: **operator ***

Introduction



- Example: **operator +**
- This example states that + symbol is overloaded.
- That is in the statement **int j = 10 + 20;** Symbol "+" works with two integers
- The statement **float j = 10.2 + 20.7;** Symbol "+" works with two floating point numbers
- The statement **Matrix m3 = m1 + m2;** Symbol "+" should work with two Matrix objects

Introduction



- **Following symbols cannot be overloaded**
- **.**
- **.***
- **⋮**
- **⋮?**
- **All other symbols can be overloaded**

Introduction



- You cannot change the **arity** of an operator
- That is the number of operands an operator takes
- Overloaded unary operators remain unary operators
- Overloaded binary operators remain binary operators
- Operators $*$ $+$ and $-$ can have both unary and binary versions
- Unary and binary versions can be separately overloaded

Introduction



- You cannot create new operators
- Only existing operators can be overloaded
- The meaning of how an operator works on values of fundamental types cannot be changed by operator overloading
- For example, you cannot overload + to perform subtraction on two int data types
- Operator overloading work with
 - Objects only that is: `Matrix m3 = m1 + m2;`
 - Object and built-in data type: `Matrix m3 = m1 + 1;`
 - Mixture of an object of a user defined data type:
 - `Matrix m3; Matrix result; Ratio r; result = m3 + r;`



Operator Overloading Involving Two Objects





Examples Binary Operator



Example - 01



```
class Matrix
{
    private:
        int rows;
        int columns;
        int **data;
    public:
        Matrix(int r, int c): rows(r), columns(c)
        {
            data = new int *[rows];
            for( int i = 0; i < rows; i++ )
            {
                data[i] = new int[columns];
                for(int j = 0; j < columns; j++ )
                    data[i][j] = 0;
            }
        }
};
```

Example - 01



```
int main(int argc, char *argv[])
{
    Matrix m1;
    Matrix m2;
    // Assume all 1's in m1 and all 2's in m2

    Matrix m3 = m1 + m2;
    // This should result in m3 with all 3's
    return 0;
}
```

Example - 01



```
Matrix operator+(const Matrix& m2) const
{
    Matrix m3(rows, columns);
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < columns; j++)
        {
            m3.data[i][j] = this->data[i][j] + m2.data[i][j];
        }
    }
    return m3;
}
```

Example - 01



- Meaning of const at the end of the operator+ function:
- Means that the member function is a const member function
- The **const** keyword at the end of the function declaration indicates that the function does not modify the state of the object on which it is called.
- In other words, the function guarantees not to change any member variables of the object.
- The function can only call other const member functions and can only access the data members in a read-only fashion.
- This is useful when you want to ensure that certain functions are only meant to inspect the object's state without modifying it.

Example - 01



- **Matrix operator+(const Matrix& m2):** The overloaded + operator takes another Matrix object m2 as a constant reference, meaning that the function cannot modify m2.
- The const at the end of the function declaration means that the operator+ function does not modify the state of the Matrix object on which it is called.



Example - 01

- Where are two matrices in the function input?
- They are presented implicitly.
- First matrix object "this" is to the left of "+" symbol
- Second matrix object is "m2" to the right of "+" symbol
- Now the statement is complete
- Matrix $m3 = m1 + m2$;



Examples Binary Operator



Example - 02



```
int main(int argc, char *argv[])
{
    // Assume all 1's in m1
    Matrix m1;

    // Read matrix from standard input
    cin >> m1;

    // write matrix to standard output
    cout << m1;

}
```

Example - 03



```
friend ostream& operator<<(ostream& output, const Matrix& m)
{
    for (int i = 0; i < m.rows; i++)
    {
        for (int j = 0; j < m.columns; j++)
        {
            output << m.data[i][j] << " ";
        }
        output << endl;
    }
    return output;
}
```

Example - 04



```
friend istream& operator>>(istream& input, const Matrix& m)
{
    for (int i = 0; i < m.rows; i++)
    {
        for (int j = 0; j < m.columns; j++)
        {
            input >> m.data[i][j];
        }
    }
    return output;
}
```

Example – 03 & 04



- **Why friend should be used for ostream and istream?**
- **These operators need direct access to the private or protected data members of a class.**
- **Member functions of a class have access to private and protected members of that class.**
- **However, non-member functions do not have such access unless they are declared as friends.**



Examples Unary Operator



Example – 05 – Prefix operator



```
Matrix& operator++()  
{  
    for (int i = 0; i < rows; i++)  
    {  
        for (int j = 0; j < columnss; j++)  
        {  
            ++data[i][j];  
        }  
    }  
    return *this;  
}
```

Example – 06 – Suffix operator



```
Matrix operator++(int)
{
    // Copy of current Matrix object
    Matrix temp = *this;
    for (int i = 0; i < rows; ++i)
    {
        for (int j = 0; j < columns; ++j)
        {
            data[i][j]++;
        }
    }
    return temp;
}
```

Example – 06 – Prefix operator



```
int main(int argc, char *argv[])
{
    Matrix mat1(2, 2);
    Matrix mat2(2, 2);

    cout << "Enter mat1 of size 2x2" << endl;
    cin >> mat1;

    cout << "Enter mat2 of size 2x2" << endl;
    cin >> mat2;

    cout << mat1 << endl;
    cout << mat2 << endl;

    cout << "Prefix operation" << endl;
    ++mat1;
    cout << mat1 << endl;

    cout << "Suffix operation" << endl;
    mat1++;
    cout << mat1 << endl;

    return 0;
}
```



Examples () Operator





Example – 07 – Suffix operator

```
int& operator() (int r, int c)
{
    return data[r][c];
}
```

```
int main(int argc, char *argv[])
{
    Matrix mat(2, 2);
    mat(0, 0) = 1; mat(0, 1) = 0;
    mat(1, 0) = 0; mat(1, 1) = 1;

    cout << mat << endl;
    return 0;
}
```

Example - 07



- The operator() function is a reference (int&) to the matrix element, **allowing modification**.
- The statement `mat(0, 0) = 1;`
- Two parameters to `operator(int r, int c)` are `r = 0` and `c = 0`;
- The righthand side value 1 is assigned to `data[r][c]` and returned



Overloading [] operator



Example - 08



- The assignment statement `mat(0, 0) = 1` is intuitive
- How about exact C array like statement `mat[0][0] = 1`?
- **mat** is the Matrix object.
- The operator `[ ]` should ensure assignment of 1 (RHS value) to the member "data" in Matrix class



Example – 08 – []operator

```
int* operator[](int r)
{
    if (r >= 0 && r < rows)
        return data[r];
    else
        throw out_of_range("Matrix row index out of range.");
}
```

```
int main(int argc, char *argv[])
{
    Matrix mat(2, 2);
    Mat[0][0] = 1; mat[0][1] = 0;
    Mat[1][0] = 0; mat[1][1] = 1;

    cout << mat << endl;
    return 0;
}
```

Example - 08



- Consider the statement `mat[0][0] = 1;`
- The first operator call `mat[0]`:
 - This calls the overloaded `operator[]` with the row index 0
 - The function checks if 0 is a valid row index
 - The function returns `data[0]`, which is a pointer to the first element of the first row.
 - So, `mat[0]` is equivalent to `data[0]`, which is a pointer to the first element of the first row.



Example - 08

- Consider the statement `mat[0][0] = 1;`
- The second operator call `mat[0][0]`:
 - Now that `mat[0]` has returned a pointer to `data[0]` (the first row), the expression `mat[0][0]` is equivalent to `data[0][0]`.
 - `data[0][0]` is the first element of the matrix
- **Assignment:** `mat[0][0] = 1;` The value 1 is assigned to `data[0][0]`.



Overloading on Objects and built-in data types



Example – 09 – Suffix operator



```
Matrix operator+(int value) const
{
    Matrix result(rows, cols);
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            result.data[i][j] = data[i][j] + value;
        }
    }
    return result;
}
```

```
int main(int argc, char *argv[])
{
    Matrix mat(2, 2);
    mat(0, 0) = 1; mat(0, 1) = 0;
    mat(1, 0) = 0; mat(1, 1) = 1;

    Matrix m = mat + 1;

    cout << mat << endl;
    return 0;
}
```



Overloading on Mixed Objects



Example - 10



- Say you have a matrix class
- Whose elements are ratios (that is user data type)

- The matrix is of the form

$\frac{1}{2}$	$\frac{3}{4}$
$\frac{1}{3}$	$\frac{2}{3}$

- Each element is a user defined data type ratio

- Now Define two objects: `Matrix m1; Ratio r1(1, 4) // 1/4;`

- `Matrix m2 = m1 + r1;` should result in m2

$\frac{3}{4}$	$\frac{1}{1}$
$\frac{7}{12}$	$\frac{11}{12}$

Example – 10



```
class Ratio
{
private:
    int n;
    int d;
public:
    Ratio(int nu = 0, int de = 1) : n(nu), d(de)
    {
        simplify();
    }
    void simplify();
    void print() const;
    friend Ratio operator+(const Ratio& r1, const Ratio& r2);
};
```


Example – 10



```
void Ratio::simplify()
{
    int gcd = std::gcd(n, d);
    n /= gcd;
    d /= gcd;
    if (d < 0)
    {
        n = -n;
        d = -d;
    }
}

void Ratio::print()
{
    cout << n << "/" << d << endl;
}
```

Example – 10



```
friend Ratio Ratio::operator+(const Ratio& r1, const Ratio& r2)
{
    int new_n = r1.n * r2.d + r2.n * r1.d;
    int new_d = r1.d * r2.d;
    return Ratio(new_n, new_d);
}
```

Example – 10



```
class Matrix
{
private:
    int rows;
    int columns;
    Ratio** data;

public:
    Matrix(int r, int c);
    ~Matrix();
    void assign(int r, int c, int nu, int de);
    Matrix operator+(const Ratio& r) const;
    friend ostream& operator<<(ostream& os, const Matrix& matrix);
};
```

Example – 10



```
Matrix::Matrix(int r, int c) : rows(r), columns(c)
{
    data = new Ratio*[rows];
    for (int i = 0; i < rows; i++)
    {
        data[i] = new Ratio[columns];
    }
}

Matrix::~~Matrix()
{
    for (int i = 0; i < rows; i++)
    {
        delete[] data[i];
    }
    delete[] data;
}
```

Example – 10



```
void Matrix::assign(int r, int c, int nu, int de)
{
    if (r >= 0 && r < rows && c >= 0 && c < columns)
    {
        data[r][c] = Ratio(nu, de);
    }
}

Matrix Matrix::operator+(const Ratio& r) const
{
    Matrix result(rows, columns);
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < columns; j++)
        {
            result.data[i][j] = data[i][j] + r;
        }
    }
    return result;
}
```

Example – 10



```
friend ostream& Matrix::operator<<(ostream& os, const Matrix& matrix)
{
    for (int i = 0; i < matrix.rows; i++)
    {
        for (int j = 0; j < matrix.columns; j++)
        {
            os << matrix.data[i][j].n << "/" << matrix.data[i][j].d << " ";
        }
        os << endl;
    }
    return os;
}
```

Example – 10



```
int main(int argc, char *argv[])
{
    Matrix mat(2, 2);
    mat.assign(0, 0, 1, 2);
    mat.assign(0, 1, 3, 4);
    mat.assign(1, 0, 1, 3);
    mat.assign(1, 1, 2, 3);

    Ratio r(1, 4);

    Matrix result = mat + r;

    cout << "Result Matrix after adding 1/4:" << endl;
    cout << result;

    return 0;
}
```

Example – 10



```
saradhi@Betaal:~/courses/cs204/july-dec-2024/tutorials/04$ g++ mixed-overload.cc -g
saradhi@Betaal:~/courses/cs204/july-dec-2024/tutorials/04$ ./a.out
Original Matrix:
1/2 3/4
1/3 2/3
Result Matrix after adding 1/4:
3/4 1/1
7/12 11/12
saradhi@Betaal:~/courses/cs204/july-dec-2024/tutorials/04$
```


Order of Evaluation



- The order of evaluation of expressions involving overloaded operators is identical to that of expressions involving built-in operators.
- The precedence and associativity of the operators are preserved when you overload them.
- **Operator Precedence:**
 - When you overload an operator, the precedence of that operator remains the same as its built-in counterpart.
 - For example, the * (multiplication) operator has higher precedence than the + (addition) operator, whether they are overloaded or not.

Order of Evaluation



- **Operator Associativity:**
 - Associativity refers to the order in which operators of the same precedence are evaluated.
 - For example, the $+$ operator is left-associative, meaning that in an expression like $a + b + c$, the operations are performed from left to right $((a + b) + c)$.
 - This associativity is preserved when operators are overloaded
- **Order of Evaluation:**
 - The order of evaluation refers to the sequence in which operands are evaluated in an expression.
 - Even though operator precedence and associativity are preserved with overloaded operators, the order in which the operands are evaluated can depend on the implementation details of the compiler.
 - For example, in an expression $a + b$, the compiler might evaluate a before b , or vice versa, depending on various factors like optimization settings.
 - This behavior is the same for both built-in and overloaded operators.



Questions?





Thank You

